A Tutorial on Vector Approximate Message Passing

Chenmin Sha

May 5, 2026

Abstract

In this tutorial, we will provide the derivation of the Vector Approximate Message Passing (VAMP) algorithm based on sum-product framework. Besides, we provide the denoiser for several sparse prior distribution. To understand the derivation in this tutorial, it is recommended to have a basic understanding of factor graph and sum-product message passing.

This tutorial does not focus on the rigorous theory of VAMP, but instead focuses on the intuition and quick implementation of the algorithm. For more rigorous theory, please refer to the original paper by Rangan et al. [1] and the follow-up works.

1 Introduction

The VAMP algorithm is to solve the following linear inverse problem:

$$\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{w} \tag{1}$$

where $\mathbf{y} \in \mathbb{C}^M$ is the observed vector, $\mathbf{A} \in \mathbb{C}^{M\times N}$ is the known measurement matrix, $\mathbf{x} \in \mathbb{C}^N$ is the unknown signal vector we want to estimate, and $\mathbf{w} \in \mathbb{C}^M$ is the noise vector. The VAMP algorithm is an iterative method that approximates the posterior distribution of $\mathbf{x}$ given $\mathbf{y}$ and $\mathbf{A}$, and it is particularly effective when $\mathbf{A}$ has certain properties (e.g., right-rotationally invariant).

It is assumed that $\mathbf{x} \sim p(\mathbf{x})$ and $\mathbf{w} \sim \mathcal{CN}(\mathbf{0}, \gamma_w^{-1}\mathbf{I})$. The goal is to compute the posterior distribution $p(\mathbf{x}|\mathbf{y}, \mathbf{A})$ and to find the MMSE estimate of $\mathbf{x}$.

2 EP based Derivation

In this section, we will derive the framework for VAMP based on sum-product message passing and expectation propagation (EP). The VAMP algorithm can be viewed as an instance of EP applied to the factor graph representation of the linear model.

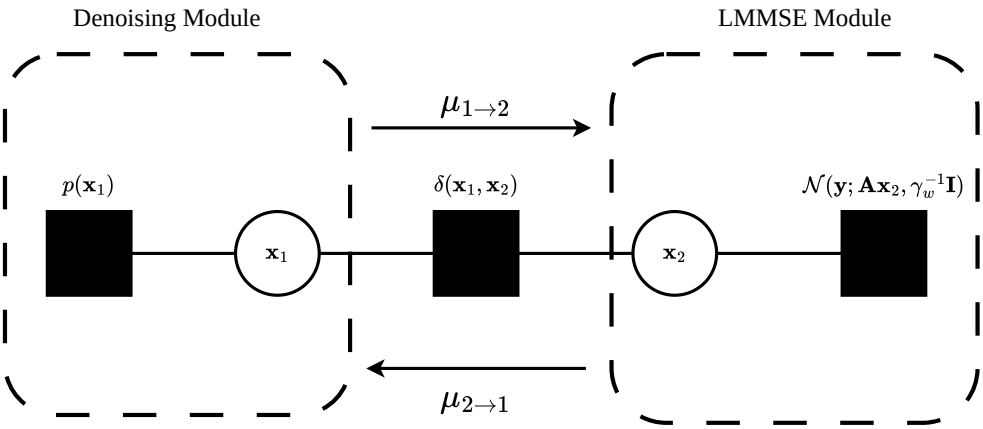

Figure 1: Factor graph representation of the linear model.

To study the joint distribution of $\mathbf{x}$ and $\mathbf{y}$, we introduce auxiliary variables $\mathbf{x}_1, \mathbf{x}_2$ that satisfies $\mathbf{x} = \mathbf{x}_1 = \mathbf{x}_2$, and study the joint distribution of $p(\mathbf{y}, \mathbf{x}_1, \mathbf{x}_2)$ instead, which can be factorized as:

$$\begin{aligned} p(\mathbf{x}_1, \mathbf{x}_2|\mathbf{y}) &\propto p(\mathbf{y}, \mathbf{x}_1, \mathbf{x}_2) \\ &= p(\mathbf{y}|\mathbf{x}_2)p(\mathbf{x}_1)\delta(\mathbf{x}_1 - \mathbf{x}_2) \\ &= p(\mathbf{x}_1)\delta(\mathbf{x}_1 - \mathbf{x}_2)\mathcal{CN}(\mathbf{y}; \mathbf{A}\mathbf{x}_2, \gamma_w^{-1}\mathbf{I}) \end{aligned} \tag{2}$$

In this way, all prior information all constraints on $\mathbf{x}$ are absorbed into $\mathbf{x}_1$, and only $\mathbf{x}_2$ is directly related to the linear measurement. The factor graph representation of the above factorization is shown in Fig. 1, where the two variable nodes $\mathbf{x}_1$ and $\mathbf{x}_2$ are connected by a delta function factor node, and $\mathbf{x}_1$ is connected to the prior factor node $p(\mathbf{x}_1)$, while $\mathbf{x}_2$ is connected to the likelihood factor node $p(\mathbf{y}|\mathbf{x}_2)$.

2.1 An overview of message passing

It would be helpful to first construct a quick view on which messages we need to handle in the iteration of the VAMP algorithm. We denote the message from variable node $\mathbf{x}_1$ to the delta function factor node as $\mu_{\mathbf{x}_1 \to \delta}(\mathbf{x}_1)$, and the message from variable node $\mathbf{x}_2$ to the delta function factor node as $\mu_{\mathbf{x}_2 \to \delta}(\mathbf{x}_2)$.

Under EP framework, all messages are approximated by Gaussian distributions. Besides, the vector Gaussian distributions should have covariance matrices in the form of a scaled identity matrix. Therefore, we can write the messages as:

$$\begin{aligned} \mu_{\mathbf{x}_1 \to \delta}(\mathbf{x}_1) &= \mathcal{CN}(\mathbf{x}_1; \mathbf{r}_1, \gamma_1^{-1}\mathbf{I}) \\ \mu_{\mathbf{x}_2 \to \delta}(\mathbf{x}_2) &= \mathcal{CN}(\mathbf{x}_2; \mathbf{r}_2, \gamma_2^{-1}\mathbf{I}) \end{aligned} \tag{3}$$

where $\mathbf{r}_1$ and $\mathbf{r}_2$ are the means of the messages, and $\gamma_1^{-1}\mathbf{I}$ and $\gamma_2^{-1}\mathbf{I}$ are the covariance matrices of the messages. Then the core of the VAMP algorithm is how to update the mean and variance parameters of the messages in each iteration.

In the above factorization, the part of $\mathbf{x}_1$ is called denoising module, since it only involves the prior distribution and the delta function constraint, while the part of $\mathbf{x}_2$ is called linear estimation module, since it involves the linear measurement and the delta function constraint. The VAMP algorithm iteratively updates the messages between these two modules until convergence — in this way, both the prior information and the measurement information are effectively utilized to estimate $\mathbf{x}$.

2.2 Derivation of EP updates

Now we are ready to derive the EP updates for the messages in the VAMP algorithm. We will first derive the update for the message $\mu_{\mathbf{x}_1 \to \delta}(\mathbf{x}_1)$, and then derive the update for the message $\mu_{\mathbf{x}_2 \to \delta}(\mathbf{x}_2)$. Since the delta function factor node will not change the information, so we can simplify the message into $\mu_{1\to 2}(\mathbf{x})$ and $\mu_{2\to 1}(\mathbf{x})$, which are the messages from module 1 to module 2 and from module 2 to module 1, respectively. This is also shown in Fig. 1.

We first derive the update for $\mu_{1\to 2}(\mathbf{x})$, and we assume that in the last interation, we have the message $\mu_{2\to 1}(\mathbf{x}) = \mathcal{CN}(\mathbf{x}; \mathbf{r}_2, \gamma_2^{-1}\mathbf{I})$. We need to notice that, if we strictly follow the sum-product message passing rule, the message $\mu_{1\to 2}(\mathbf{x})$ should just be $p(\mathbf{x})$. However, this message is not Gaussian, and it cannot be directly used in the next iteration. Therefore, we need to apply EP to approximate this message by a Gaussian distribution. Here, we do not follow the standard EP procedure, but instead understand why we need to obtain the Gaussian approximation of the message in this manner.

The most important point is that, if we simple approximate $p(\mathbf{x})$ using Gaussian, then it lacks the information from the last iteration, which is contained in the message $\mu_{2\to 1}(\mathbf{x})$. Therefore, consider Gaussian approximation from the following perspective:

$$\mu_{1\to 2}(\mathbf{x}) = \frac{p(\mathbf{x})\mu_{2\to 1}(\mathbf{x})}{\mu_{2\to 1}(\mathbf{x})} \approx \frac{\mathcal{CN}(\mathbf{x}; \hat{\mathbf{x}}, \eta^{-1}\mathbf{I})}{\mathcal{CN}(\mathbf{x}; \mathbf{r}_2, \gamma_2^{-1}\mathbf{I})} = \mathcal{CN}(\mathbf{x}; \mathbf{r}_1, \gamma_1^{-1}\mathbf{I})$$

That is, we first compute the product of the prior distribution and the incoming message $\mu_{2\to 1}(\mathbf{x})$, which gives us a new distribution that incorporates both the prior information and the information from the last iteration. Then we approximate this new distribution by a Gaussian distribution, and finally we divide by the incoming message $\mu_{2\to 1}(\mathbf{x})$ to obtain the outgoing message $\mu_{1\to 2}(\mathbf{x})$. This way, the outgoing message $\mu_{1\to 2}(\mathbf{x})$ effectively combines the prior information and the information from the last iteration, which is crucial for the convergence of the VAMP algorithm.

How to compute the mean $\hat{\mathbf{x}}$ and the variance η^{-1} of the Gaussian approximation is left for the next section, and now we further derive other updates in the VAMP algorithm. After the above approximation, the message $\mu_{1\to 2}(\mathbf{x})$ is now a Gaussian distribution with the following mean and covariance:

$$\mathbf{r}_1 = \frac{\hat{\mathbf{x}}\eta - \mathbf{r}_2\gamma_2}{\eta - \gamma_2}, \quad \gamma_1 = \eta - \gamma_2 \tag{4}$$

Then at the LMMSE side, it will receive the message $\mu_{1\to 2}(\mathbf{x})$ from the denoising module, and it will compute the message $\mu_{2\to 1}(\mathbf{x})$ to send back to the denoising module. The update for $\mu_{2\to 1}(\mathbf{x})$ can be derived in a similar way, i.e., we first approximate the posterior distribution of $\mathbf{x}$ given the linear measurement and the incoming message $\mu_{1\to 2}(\mathbf{x})$ to our desired Gaussian form, and then we divide by the incoming message $\mu_{1\to 2}(\mathbf{x})$ to obtain the outgoing message $\mu_{2\to 1}(\mathbf{x})$.

This step is easier as the posterior distribution of $\mathbf{x}$ given the linear measurement and the incoming message $\mu_{1\to 2}(\mathbf{x})$ is already Gaussian, and we can directly compute its mean and covariance. The only approximation here is we need the message to be independent across different dimensions and have identical variance, which can be achieved by simply averaging the diagonal elements of the true covariance matrix. So the message update rule for $\mu_{2\to 1}(\mathbf{x})$ can be written as:

1. Compute the posterior distribution of $\mathbf{x}$ given the linear measurement and the incoming message $\mu_{1\to 2}(\mathbf{x})$, which is a Gaussian distribution with mean $\hat{\mathbf{x}}$ and covariance $\mathbf{C}$. And we can directly compute them:

$$\hat{\mathbf{x}} = \mathbf{r}_1 + \gamma_1^{-1}\mathbf{A}^H\left(\gamma_w^{-1}\mathbf{I} + \gamma_1^{-1}\mathbf{A}\mathbf{A}^H\right)^{-1}(\mathbf{y} - \mathbf{A}\mathbf{r}_1) \tag{5}$$

$$\mathbf{C} = \gamma_1^{-1}\mathbf{I} - \gamma_1^{-2}\mathbf{A}^H\left(\gamma_w^{-1}\mathbf{I} + \gamma_1^{-1}\mathbf{A}\mathbf{A}^H\right)^{-1}\mathbf{A} \tag{6}$$

   Note that this step is equivalent to computing the LMMSE estimator of $\mathbf{x}$ given the linear measurement $\mathbf{y}$ under prior distribution $\mathcal{CN}(\mathbf{x}; \mathbf{r}_1, \gamma_1^{-1}\mathbf{I})$, so this is called the linear estimation module in the VAMP algorithm.

2. Approximate the covariance matrix $\mathbf{C}$ by a scaled identity matrix, i.e., $\mathbf{C} \approx \eta^{-1}\mathbf{I}$, where η can be computed as $\eta = \left(\frac{1}{N}\mathrm{Tr}(\mathbf{C})\right)^{-1}$.

3. We update the message $\mu_{2\to 1}(\mathbf{x}) = \mathcal{CN}(\mathbf{x}; \mathbf{r}_2, \gamma_2^{-1}\mathbf{I})$, which is computed as:

$$\mathbf{r}_2 = \frac{\hat{\mathbf{x}}\eta - \mathbf{r}_1\gamma_1}{\eta - \gamma_1}, \quad \gamma_2 = \eta - \gamma_1 \tag{7}$$

2.3 SVD based LMMSE Estimator

In this section, we show how SVD can help to compute the LMMSE estimator (5) in the VAMP algorithm efficiently. This is important to alliviate the computational burden of matrix inversion in the LMMSE estimator — and I think this is the main reason why VAMP can be mush faster than Sparse Bayesian Learning (SBL) algorithm, which is also a powerful sparse recovery algorithm. The SBL framework also relies on the LMMSE estimator but cannot utilize SVD to speed up the computation.

We denote the SVD of $\mathbf{A}$ as $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^H$, where $\mathbf{U} \in \mathbb{C}^{M\times N}$ and $\mathbf{V} \in \mathbb{C}^{N\times N}$ and $\mathbf{\Sigma} \in \mathbb{C}^{M\times N}$. That is, we need $\mathbf{U}$ to span the whole space of $\mathbb{C}^M$, and $\mathbf{V}$ also spans the space of $\mathbb{C}^N$ — we take SVD in this form just to simplify the following derivation. And then we have:

$$\begin{aligned}
\hat{\mathbf{x}} &= \mathbf{r}_1 + \gamma_1^{-1}\mathbf{A}^H\left(\gamma_w^{-1}\mathbf{I} + \gamma_1^{-1}\mathbf{A}\mathbf{A}^H\right)^{-1}(\mathbf{y} - \mathbf{A}\mathbf{r}_1) \\
&= \mathbf{r}_1 + \gamma_1^{-1}\mathbf{V}\mathbf{\Sigma}^H\mathbf{U}^H\left(\gamma_w^{-1}\mathbf{I} + \gamma_1^{-1}\mathbf{U}\mathbf{\Sigma}\mathbf{\Sigma}^H\mathbf{U}^H\right)^{-1}(\mathbf{y} - \mathbf{U}\mathbf{\Sigma}\mathbf{V}^H\mathbf{r}_1) \\
&= \mathbf{r}_1 + \gamma_1^{-1}\mathbf{V}\mathbf{\Sigma}^H\left(\gamma_w^{-1}\mathbf{I} + \gamma_1^{-1}\mathbf{\Sigma}\mathbf{\Sigma}^H\right)^{-1}(\mathbf{U}^H\mathbf{y} - \mathbf{\Sigma}\mathbf{V}^H\mathbf{r}_1) \\
&= \gamma_w\mathbf{V}(\gamma_1\mathbf{I} + \gamma_w\mathbf{\Sigma}^H\mathbf{\Sigma})^{-1}\mathbf{\Sigma}^H\mathbf{U}^H\mathbf{y} + \gamma_1\mathbf{V}(\gamma_1\mathbf{I} + \gamma_w\mathbf{\Sigma}^H\mathbf{\Sigma})^{-1}\mathbf{V}^H\mathbf{r}_1 \\
\mathbf{C} &= \gamma_1^{-1}\mathbf{I} - \gamma_1^{-2}\mathbf{A}^H\left(\gamma_w^{-1}\mathbf{I} + \gamma_1^{-1}\mathbf{A}\mathbf{A}^H\right)^{-1}\mathbf{A} \\
&= \gamma_1^{-1}\mathbf{I} - \gamma_1^{-2}\mathbf{V}\mathbf{\Sigma}^H\mathbf{U}^H\left(\gamma_w^{-1}\mathbf{I} + \gamma_1^{-1}\mathbf{U}\mathbf{\Sigma}\mathbf{\Sigma}^H\mathbf{U}^H\right)^{-1}\mathbf{U}\mathbf{\Sigma}\mathbf{V}^H \\
&= \gamma_1^{-1}\mathbf{I} - \gamma_1^{-1}\gamma_w\mathbf{V}\mathbf{\Sigma}^H\mathbf{\Sigma}(\gamma_1\mathbf{I} + \gamma_w\mathbf{\Sigma}^H\mathbf{\Sigma})^{-1}\mathbf{V}^H \\
\mathrm{Tr}(\mathbf{C}) &= \gamma_1^{-1}N - \gamma_1^{-1}\gamma_w\sum_{i=1}^{N}\frac{\sigma_i^2}{\gamma_1 + \gamma_w\sigma_i^2}
\end{aligned} \tag{8}$$

The above transformation can largely reduce the computational complexity of the LMMSE estimator, as the matrix inversion is now reduced to the inversion of a diagonal matrix, which can be computed efficiently. And in practice, we may not need to compute the full SVD of $\mathbf{A}$, but only need to compute the non-zero singular values and the corresponding singular vectors.

2.4 Summation of the algorithm

We now summarize the full procedure of VAMP.

Algorithm 1 VAMP Algorithm.

Input: $\mathbf{y}, \mathbf{A}, \gamma_w$, prior $p(\mathbf{x})$, max iteration $T_{\max}$, tolerance ϵ.

Initialization: choose $\mathbf{r}_2^{(0)}$ (e.g., $\mathbf{0}$), $\gamma_2^{(0)} > 0$, and set $t = 0$.

(Optional, for fast LMMSE) precompute SVD $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^H$ and $\tilde{\mathbf{y}} = \mathbf{U}^H\mathbf{y}$.

for $t = 0, 1, \ldots, T_{\max} - 1$ **do**

Module 1 (denoising):

Form the belief

$$q_1^{(t)}(\mathbf{x}) \propto p(\mathbf{x})\, \mathcal{CN}(\mathbf{x}; \mathbf{r}_2^{(t)}, (\gamma_2^{(t)})^{-1}\mathbf{I}).$$

Compute posterior mean and scalar precision

$$\hat{\mathbf{x}}_1^{(t)} = \mathbb{E}_{q_1^{(t)}}[\mathbf{x}], \quad \eta_1^{(t)} = \left(\frac{1}{N}\operatorname{Tr}\left(\operatorname{Cov}_{q_1^{(t)}}(\mathbf{x})\right)\right)^{-1}.$$

Extrinsic update to Module 2

$$\gamma_1^{(t)} = \eta_1^{(t)} - \gamma_2^{(t)}, \quad \mathbf{r}_1^{(t)} = \frac{\eta_1^{(t)}\hat{\mathbf{x}}_1^{(t)} - \gamma_2^{(t)}\mathbf{r}_2^{(t)}}{\gamma_1^{(t)}}.$$

Module 2 (linear MMSE):

Compute

$$\hat{\mathbf{x}}_2^{(t)} = \mathbf{r}_1^{(t)} + (\gamma_1^{(t)})^{-1}\mathbf{A}^H\left(\gamma_w^{-1}\mathbf{I} + (\gamma_1^{(t)})^{-1}\mathbf{A}\mathbf{A}^H\right)^{-1}(\mathbf{y} - \mathbf{A}\mathbf{r}_1^{(t)}),$$

$$\mathbf{C}_2^{(t)} = (\gamma_1^{(t)})^{-1}\mathbf{I} - (\gamma_1^{(t)})^{-2}\mathbf{A}^H\left(\gamma_w^{-1}\mathbf{I} + (\gamma_1^{(t)})^{-1}\mathbf{A}\mathbf{A}^H\right)^{-1}\mathbf{A},$$

$$\eta_2^{(t)} = \left(\frac{1}{N}\operatorname{Tr}(\mathbf{C}_2^{(t)})\right)^{-1}.$$

Extrinsic update to Module 1

$$\gamma_2^{(t+1)} = \eta_2^{(t)} - \gamma_1^{(t)}, \quad \mathbf{r}_2^{(t+1)} = \frac{\eta_2^{(t)}\hat{\mathbf{x}}_2^{(t)} - \gamma_1^{(t)}\mathbf{r}_1^{(t)}}{\gamma_2^{(t+1)}}.$$

Stopping rule: if

$$\frac{\|\hat{\mathbf{x}}_2^{(t)} - \hat{\mathbf{x}}_2^{(t-1)}\|_2}{\|\hat{\mathbf{x}}_2^{(t)}\|_2 + 10^{-12}} < \epsilon,$$

then stop.

Numerical safeguard: if any $\gamma_1^{(t)}$ or $\gamma_2^{(t+1)} \leq 0$, apply damping/clipping to keep precisions positive.

end for

Output: $\hat{\mathbf{x}} = \hat{\mathbf{x}}_2^{(t)}$.

---

3 Closed Form Update under Certain Prior Distributions

In this section, we will show how to compute the mean $\hat{\mathbf{x}}$ and the variance η^{-1} of the Gaussian approximation in the denoising module under certain prior distributions. This is important for the implementation of the VAMP algorithm, as it allows us to efficiently compute the updates in each iteration.

In this tutorial, we will focus on the block sparse signals, where the prior distribution is assumed to be a Bournoulli-Gaussian distribution, i.e., each block of $\mathbf{x}$ is independently drawn from a mixture of a point mass at zero and a Gaussian distribution:

$$p(\mathbf{x}_i) = (1-\lambda)\delta(\mathbf{x}_i) + \lambda\mathcal{CN}(\mathbf{x}_i; \mathbf{0}, \sigma_x^2\mathbf{I}) \tag{9}$$

where there are a total of G blocks:

$$\mathbf{x} = (\mathbf{x}_1^\top, \cdots, \mathbf{x}_G^\top)^\top$$

Assume that the incoming message $\mathcal{CN}(\mathbf{x}; \mathbf{r}_2, \gamma_2^{-1}\mathbf{I})$, then we try to compute the Gaussian approximation of the posterior distribution $p(\mathbf{x})\mathcal{CN}(\mathbf{x}; \mathbf{r}_2, \gamma_2^{-1}\mathbf{I})$ — or strictly speaking, we need to compute the mean and variance of this distribution. And here, the posterior distribution can be interpreted as we obtain the observation:

$$\mathbf{r}_2 = \mathbf{x} + \mathbf{w}, \quad \mathbf{w} \sim \mathcal{CN}(\mathbf{0}, \gamma_2^{-1}\mathbf{I})$$

First, note that the posterior is actually independent for different blocks. For the i-th block $\mathbf{x}_i$ (of length d), the posterior (up to nomalization) is:

$$(1-\lambda)\delta(\mathbf{x}_i)\mathcal{CN}(\mathbf{x}_i; \mathbf{r}_{2,i}, \gamma_2^{-1}\mathbf{I}) + \lambda\mathcal{CN}(\mathbf{x}_i; \mathbf{0}, \sigma_x^2\mathbf{I})\mathcal{CN}(\mathbf{x}_i; \mathbf{r}_{2,i}, \gamma_2^{-1}\mathbf{I}) \tag{10}$$

It is still a Bournoulli-Gaussian distribution in the form:

$$(1-\pi_i)\delta(\mathbf{x}_i) + \pi_i\mathcal{CN}(\mathbf{x}_i; \boldsymbol{\mu}_i, \mathbf{C}_i)$$

We need to first determine the parameters π_i, $\boldsymbol{\mu}_i$ and $\mathbf{C}_i$ and then the mean and covariance are easy to compute. The Gaussian part can be computed as:

$$\mathbf{C}_i = \frac{\sigma_x^2}{\gamma_2\sigma_x^2 + 1}\mathbf{I}, \quad \boldsymbol{\mu}_i = \frac{\gamma_2\sigma_x^2}{\gamma_2\sigma_x^2 + 1}\mathbf{r}_{2,i} \tag{11}$$

Then we compute the posterior active probability π_i.

$$\pi_i = p(\text{active}|\mathbf{r}_{2,i}) = \frac{\lambda p(\mathbf{r}_{2,i}|\text{active})}{(1-\lambda)p(\mathbf{r}_{2,i}|\text{inactive}) + \lambda p(\mathbf{r}_{2,i}|\text{active})}$$

And the two likelihoods can be computed as:

$$\begin{aligned}
p(\mathbf{r}_{2,i}|\text{active}) &= \int p(\mathbf{r}_{2,i}|\mathbf{x}_i)p(\mathbf{x}_i|\text{active})d\mathbf{x}_i = \mathcal{CN}(\mathbf{r}_{2,i}; \mathbf{0}, (\sigma_x^2 + \gamma_2^{-1})\mathbf{I}) \\
p(\mathbf{r}_{2,i}|\text{inactive}) &= \int p(\mathbf{r}_{2,i}|\mathbf{x}_i)p(\mathbf{x}_i|\text{inactive})d\mathbf{x}_i = \mathcal{CN}(\mathbf{r}_{2,i}; \mathbf{0}, \gamma_2^{-1}\mathbf{I})
\end{aligned}$$

Then we have:

$$\pi_i = \frac{\lambda\mathcal{CN}(\mathbf{r}_{2,i}; \mathbf{0}, (\sigma_x^2 + \gamma_2^{-1})\mathbf{I})}{(1-\lambda)\mathcal{CN}(\mathbf{r}_{2,i}; \mathbf{0}, \gamma_2^{-1}\mathbf{I}) + \lambda\mathcal{CN}(\mathbf{r}_{2,i}; \mathbf{0}, (\sigma_x^2 + \gamma_2^{-1})\mathbf{I})}$$

Now we can compute the posterior mean and covariance of $\mathbf{x}_i$:

$$\hat{\mathbf{x}}_i = \pi_i\boldsymbol{\mu}_i, \quad \text{Cov}(\mathbf{x}_i) = \pi_i\mathbf{C}_i + \pi_i(1-\pi_i)\boldsymbol{\mu}_i\boldsymbol{\mu}_i^H \tag{12}$$

And finally we can update our message:

$$\begin{aligned}
\eta &= \left(\frac{1}{N}\sum_{i=1}^{G}\text{Tr}(\text{Cov}(\mathbf{x}_i))\right)^{-1} \\
&= \left(\frac{\pi_i\sigma_x^2}{\gamma_2\sigma_x^2 + 1} + \frac{1}{N}\sum_{i=1}^{G}\frac{\pi_i(1-\pi_i)\gamma_2^2\sigma_x^4\|\mathbf{r}_{2,i}\|^2}{(\gamma_2\sigma_x^2 + 1)^2}\right)^{-1} \\
\mathbf{r}_{1,i} &= \frac{\eta\hat{\mathbf{x}}_i - \gamma_2\mathbf{r}_{2,i}}{\eta - \gamma_2} \\
&= \frac{\eta\pi_i\boldsymbol{\mu}_i - \gamma_2\mathbf{r}_{2,i}}{\eta - \gamma_2} \\
\gamma_1 &= \eta - \gamma_2
\end{aligned} \tag{13}$$

References

[1] S. Rangan, P. Schniter, and A. K. Fletcher, "Vector Approximate Message Passing," June 2018. arXiv:1610.03082 [cs].